

Machine Learning for Time Series

Comprehensive Course Summary

Course: MLTS / MLTS-Deluxe (5 / 7.5 ECTS)

Institution: Friedrich-Alexander-Universität Erlangen-Nürnberg

Lab: Machine Learning and Data Analytics (MaD) Lab

Lecturer: Dr. Dario Zanca · Prof. Dr. Björn Eskofier

Lectures covered: 14 ($\approx$ 685 slides)

About this document. This is a personal study summary of the MLTS lectures, written to consolidate the core concepts, equations, intuitions and practical implications of every lecture. It is intended as a companion to the original slides and as a reference for the implementation exercises that accompany this repository.

Each lecture chapter follows the same structure: a short overview of the topic, the key concepts and equations explained in plain language, the main algorithmic ideas, and a take-home box highlighting the practical implications you should remember.

Table of Contents

#	Lecture	Topic	Page
	Course overview	Motivation and roadmap	3
1	Introduction & Time Series Fundamentals (I)	Definitions, stationarity, ergodicity	4
2	Time Series Fundamentals (II)	ML pipeline, tasks, linear regression	6
3	Bayesian Inference	Bayes' rule, priors, model selection	8
4	Gaussian Process Regression	GP priors over functions, inference	10
5	Gaussian Process Classification	GP + sigmoid, two-step prediction	12
6	State Space Models & Kalman Filtering	LG-SSM, KF, EKF, UKF	14
7	Sequential Monte Carlo & Particle Filtering	Importance sampling, SIR, particle PF	17
8	Autoregressive Models	AR, MA, ARMA, ARIMA	19
9	Data Mining with Time Series	Frequency, DTW, matrix profile	21
10	Recurrent Neural Networks	Perceptron → MLP → RNN, BPTT	23
11	Gated Models	LSTM, GRU, Phased-LSTM	25
12	Convolutional Models	1D/2D conv, TCN, ConvLSTM	27
13	Attention-based Models	Attention, Transformer, multi-head	29
14	Domain Adaptation	UDA, OOD generalization, DANN	31
	Course synthesis & cheat sheet	When to reach for which model, themes	33

Course Overview

Machine Learning for Time Series (MLTS) is built around a single guiding question: *how do we model data whose samples are not independent and identically distributed but instead arrive sequentially in time?* The course progresses from classical statistical foundations to state-of-the-art deep learning architectures, always keeping the specific challenges of temporal data in mind: dependence between observations, non-stationarity, irregular sampling, long-range memory, and the need to deal with noise and uncertainty in a principled way.

Why time-series ML matters

Time series have been studied since antiquity — the Babylonians recorded periodic planetary phenomena, and John Graunt's 17th-century weekly mortality reports laid the foundations of epidemiology. Today the explosion of sensor data (the world produced about 47 zettabytes in 2020, up from 2 zettabytes in 2010) makes machine learning on time series central in healthcare, IoT, smart cities, and process monitoring.

Roadmap of the 14 lectures

- **Foundations (L1-2):** what a time series is, properties (stationarity, ergodicity), the ML pipeline, common tasks (classification, forecasting, anomaly detection, clustering, segmentation).
- **Bayesian core (L3):** Bayes' theorem, priors, model selection via marginal likelihood (Bayesian Occam's razor), Bayesian linear regression.
- **Gaussian processes (L4-5):** putting priors directly on functions; regression and classification with GPs.
- **State-space models (L6-7):** latent-state dynamical systems with the Kalman filter (linear-Gaussian), Extended/Unscented KF (non-linear), and particle filters (non-Gaussian, Monte Carlo).
- **Classical sequence models (L8):** AR, MA, ARMA, ARIMA — workhorses of statistical forecasting.
- **Data mining (L9):** Fourier and wavelet analysis, dynamic time warping, matrix profiles, signature method, Python tooling.
- **Deep learning (L10-13):** recurrent networks (RNN, LSTM, GRU), convolutional approaches (1D-CNN, ConvLSTM, TCN) and attention (Transformer architecture).
- **Generalisation (L14):** domain adaptation and out-of-distribution generalisation — what to do when training and deployment distributions differ.

How to use this summary

Read the cover questions first, then skim the equations, then read the *take-home* box at the end of each lecture. Use the original slides for additional examples and the accompanying code repository to implement the algorithms hands-on.

Lecture 1 — Introduction & Time-Series Fundamentals (I)

This first lecture sets the stage: it motivates time-series ML through historical and modern examples, defines what a time series is, and introduces the statistical concepts (stochastic processes, stationarity, ergodicity, the central limit theorem) that the rest of the course relies on.

Definition and basic terminology

A time series is a set of observations $S = \{s_1, \dots, s_T\}$ where each $s_i \in \mathbb{R}^d$ is the measured state of some process at time t_i . The defining feature is that the samples are typically **not independent**; understanding the structure of this dependency is the central concern of the field.

Several axes describe a time series:

- **Regularly vs irregularly sampled.** Regular: constant Δt between observations. Irregular: stored as pairs (s, t) .
- **Univariate ($d = 1$) vs multivariate ($d > 1$).** A tri-axial accelerometer is a typical multivariate example.
- **Discrete vs continuous.** Continuous = observed at every instant; discrete = sampled at specific times. Mixed-type series combine both.
- **Periodic.** $\exists \tau$ such that $s_i = s_{i+\tau}$ for all valid i . A pure sine wave is periodic; a heartbeat is only approximately so.
- **Deterministic vs non-deterministic.** A deterministic series is fully described by an analytical formula; non-deterministic series are modelled as realisations of stochastic processes.

Stochastic processes

A non-deterministic time series is treated as a **realisation** of a stochastic process — a collection of random variables $\{X_t\}_{t \in \mathbb{Z}}$. Even an infinitely long observation would still be a single realisation of that process. If we had a battery of N parallel processes, the observations at time t would form a sample of N realisations of the random variable X_t .

Two summary statistics matter most. The **unconditional mean** $\mu_t = E[X_t]$ and the **j-th autocovariance**:

$$\gamma_{jt} = E[(X_t - \mu_t)(X_{t-j} - \mu_{t-j})]$$

Stationarity

A process is **(weakly) stationary** if neither its mean nor its autocovariance depend on t — i.e. the statistics are invariant under time shifts. The simplest stationary example is $X_t = \mu + \varepsilon_t$ with ε_t Gaussian white noise: the mean is constantly μ and the autocovariance depends only on the lag j , not on t .

Ergodicity

A process is **ergodic** if its time average (computed from one long realisation) converges to the process mean as $T \rightarrow \infty$. In other words, **time statistics equal ensemble statistics** for a long-enough observation. Stationarity does not imply ergodicity: if each realisation has its own random offset $\mu^{(i)}$ drawn from $N(0, \lambda^2)$, the process is stationary but its sample temporal mean converges to $\mu^{(i)}$, not to the process mean of zero.

Why i.i.d. assumptions usually fail

Most statistical tools assume independent and identically distributed samples. Time series violate this routinely: a stock price today depends on yesterday (dependence) and its volatility may change over time (non-identical distribution). We therefore measure dependence between two variables with the **covariance** $\text{Cov}(X_i, X_j)$ and the dependence of an entire series with the **long-run variance** $\sigma_i^2 = \sum_{k \in \mathbb{Z}} \text{Cov}(X_i, X_{i+k})$.

Central Limit Theorem (CLT)

For an i.i.d. sequence, the CLT guarantees that the standardised sum converges in distribution to $N(0, \sigma^2)$. For stationary time series with long-run variance σ^2 , the CLT still holds. Less restrictive versions exist for M-dependent processes (where observations more than M apart are independent). The CLT is what justifies the standard assumption of Gaussian noise in linear regression.

Take-home

Before applying any classical statistical tool, ask: *is this series stationary?* If not, transform it (differencing, detrending) until it is. The CLT, and most inference machinery built on it, depend on this. Stationarity $\neq$ ergodicity — a single long realisation only tells you the whole story when the process is ergodic.

Lecture 2 — Time-Series Fundamentals (II)

Lecture 2 zooms out to the practice of machine learning: types of learning, the standard ML pipeline, the most common time-series tasks, and a worked example of linear regression as a forecasting tool.

Three flavours of machine learning

Paradigm	Signal	Typical tasks
Supervised	Labelled (input → output) pairs	Classification, regression
Unsupervised	No labels, learning from a metric	Clustering, dimensionality reduction
Reinforcement	Reward / punishment signals from interaction	Control, sequential decision-making

A key warning is the **curse of dimensionality**: as the number of features grows, data needs explode exponentially. With 100 binary sensors there are $2^{100} \approx 1.27 \times 10^{30}$ possible states — more than the number of atoms in the human body. This is why dimensionality reduction (feature selection or extraction) is often essential.

The 5-step ML pipeline

- **1. Problem definition.** What goal? What kind of data? E.g. predict machine failure one week in advance using sensor and log data.
- **2. Data collection & preprocessing.** Ingest raw data; remove missing values and outliers; normalise; segregate into train, validation, and test splits.
- **3. Model training.** Pick an algorithm appropriate to the task and fit it on the training set.
- **4. Model evaluation.** Measure performance on the validation set, tune hyperparameters, repeat. Final unbiased number comes from the untouched test set.
- **5. Deployment & monitoring.** Real-time and hardware constraints matter; monitor for sensor drift and out-of-range data.

Evaluation metrics

Classification is summarised by the confusion matrix (TP, TN, FP, FN) and accuracy = $(TP + TN) / \text{total}$. **Regression** uses Mean Absolute Error (no error direction information) or Mean Squared Error (larger errors penalised more heavily).

$$MAE = (1/N) \sum |y_i - \hat{y}_i| \quad | \quad MSE = (1/N) \sum (y_i - \hat{y}_i)^2$$

K-fold cross-validation resamples the dataset so that every point serves both for training and validation across iterations — particularly useful when data is scarce, and a sound way to choose hyperparameters.

Common ML tasks on time series

- **Classification.** Map a whole series to a label, e.g. $f(S^{(i)}) = c^{(i)} \in \{0, 1\}^L$ (one-hot).
- **Forecasting.** Predict future values from past history.
- **Anomaly detection.** Flag observations or sub-sequences that deviate from the normal regime.
- **Segmentation.** Split a series into homogeneous sub-sequences.
- **Clustering.** Group similar series together for analysis or to drive downstream recommender systems.

Worked example: linear regression for forecasting

Linear regression is the simplest forecasting model and the building block of more complex ones (deep networks included). Given features $\mathbf{x}$ and weights $\mathbf{w}$, the model is:

$$f(\mathbf{x}) = w_0 x_0 + w_1 x_1 + \varepsilon = \sum_j w_j x_j + \varepsilon, \text{ with } \varepsilon \sim N(0, \sigma^2)$$

The parameters $\theta = (w, \sigma)$ are estimated by maximum likelihood, which for a Gaussian noise model is equivalent to minimising the negative log-likelihood:

$$NLL(\theta) = \frac{1}{2} (\mathbf{y} - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mathbf{X}\mathbf{w}), \text{ solution: } \mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

This is the closed-form least-squares solution. The same equation re-emerges in many later lectures (Bayesian linear regression in L3, GP regression in L4) — only the prior over the weights changes.

Take-home

Identify your task *before* picking the model: classification, forecasting, anomaly detection, segmentation and clustering all need different inputs, outputs and metrics. Always keep an untouched test set; the validation set is for tuning, the test set is for honesty.

Lecture 3 — Bayesian Inference

The Bayesian view is the conceptual backbone of much of the rest of MLTS. Rather than producing a single point estimate of model parameters, Bayesian inference works with full distributions, propagating uncertainty all the way to predictions.

Bayes' theorem

$$P(A | B) = P(B | A) \cdot P(A) / P(B)$$

Each term has a name worth memorising: **posterior** $P(A | B)$ — what we believe about A after observing B; **likelihood** $P(B | A)$ — how probable B is if A is true; **prior** $P(A)$ — what we believed about A before any data; **marginal** $P(B)$ — the evidence for B, which acts as a normalising constant.

Iterative example. Disease prevalence 1%, test accuracy 95%. After one positive test the posterior is 0.161. After two: 0.785. After three: 0.986. The posterior of one test becomes the prior of the next — a powerful way of accumulating evidence.

Bayesian modelling

For a parameterised model with parameters θ and observed data D , the machinery is:

$$p(\theta | D) = p(D | \theta) \cdot p(\theta) / p(D) \quad | \quad p(y | x, D) = \int p(y | \theta, x, D) \cdot p(\theta | D) d\theta$$

The first equation is Bayes' rule for parameters. The second is the **predictive distribution**: predictions are obtained by averaging over *all* plausible parameter settings, weighted by the posterior. Compared to maximum likelihood, this gives not a point but a distribution — uncertainty is preserved.

Bayesian model selection — Occam's razor

Different models M_i can be compared by their **marginal likelihood**:

$$p(D | M_i) = \int p(D | \theta, M_i) \cdot p(\theta | M_i) d\theta$$

Since $\sum_D p(D | M_i) = 1$, complex models that can explain many possible datasets must spread their probability mass thin and therefore assign less probability to any specific dataset. Bayesian inference therefore implements Occam's razor automatically: *too simple* models cannot generate the data; *too complex* models could generate too many things and end up with low evidence; the best model is the smallest one that adequately explains the data. This is far more efficient than fitting every candidate K times via cross-validation.

Priors

- **Subjective vs objective.** A subjective prior encodes the modeller's belief; an objective prior tries to be uninformative.
- **Informative vs uninformative.** An informative prior expresses a specific expectation (e.g. tomorrow's noon temperature is centred on today's noon temperature); an uninformative prior expresses ignorance (50/50 for a coin we know nothing about).
- **Hierarchical (multi-level) priors.** When the prior itself depends on hyperparameters α , you have $\alpha \rightarrow \theta \rightarrow D$. Useful when you want to pool information across groups (e.g. cancer mortality across cities).
- **Empirical priors.** A computational shortcut: estimate the hyperparameters α by maximising the marginal likelihood $p(D, \alpha)$, then plug them in. Less prone to overfitting than estimating θ directly because α is much lower-dimensional.
- **Conjugate priors.** A prior is conjugate to a likelihood if the posterior has the same algebraic form as the prior. This gives closed-form posteriors (e.g. Beta-Binomial) and makes the math tidy.

Bayesian linear regression

Assuming Gaussian noise of known variance σ^2 , the likelihood of the weights is Gaussian. The conjugate prior is also Gaussian $p(w) = N(w \mid w_0, V_0)$; applying Bayes' rule for Gaussians gives a Gaussian posterior with closed-form mean and covariance:

$$\begin{aligned} w_N &= V_N (V_0^{-1} w_0 + (1/\sigma^2) X^T y) \\ V_N &= \sigma^2 (\sigma^2 V_0^{-1} + X^T X)^{-1} \end{aligned}$$

The predictive distribution at a test point x is also Gaussian, with variance $\sigma^2(x) = \sigma^2 + x^T V_N x$ — the prediction uncertainty has two contributions: irreducible noise σ^2 and posterior uncertainty in the weights $x^T V_N x$.

Take-home

Bayesian inference replaces single point estimates with full distributions. The price is the often-intractable integral over θ ; conjugate priors and computational shortcuts (empirical Bayes, hierarchical models) keep things manageable. The beauty is that the marginal likelihood automatically prefers the simplest adequate model — Occam's razor for free.

Lecture 4 — Gaussian Process Regression

Lectures 4 and 5 take Bayesian inference a step further: instead of putting a prior on the parameters of a chosen functional form, we put a prior directly **on functions**. The vehicle for this is the Gaussian process.

Recap: Gaussian distributions

Both the marginal and the conditional of a jointly Gaussian distribution are themselves Gaussian. If

$$p(x, y) = N([a; b], [[A, B]; [B^T, C]])$$

then $p(x) = N(a, A)$ and $p(x | y) = N(a + B C^{-1}(y - b), A - B C^{-1} B^T)$. These two formulae do all the heavy lifting in GP inference.

From parameter priors to function priors

Choosing a model structure (e.g. polynomial degree) and a prior on its parameters indirectly induces a prior over functions. The natural question is: *can we specify a prior on functions directly, without committing to a parametric family?* A GP says yes.

Gaussian process — definition

A function f is a **Gaussian process** if for any finite collection of input points $t = (t_1, \dots, t_N)$, the vector $f(t) = (f(t_1), \dots, f(t_N))^T$ has a multivariate Gaussian distribution. In other words, picking any number of input points always gives a Gaussian over the corresponding outputs.

A GP is fully described by a **mean function** $\mu(t)$ (often taken to be zero after centring) and a **covariance / kernel function** $k(t, t')$. The covariance encodes how related two outputs are: if the inputs are similar according to k , their outputs will also be similar. We write $f \sim \text{GP}(\mu, k)$.

$$f(t) \sim N(\mu(t), k(t, t'))$$

GP regression: conditioning and inference

Suppose we observe y at training points and want to predict f at the same locations. Treating (f, y) as jointly Gaussian with covariance blocks $\Sigma_{ff}, \Sigma_{fy}, \Sigma_{yy}$:

$$\begin{aligned} p(f | y) &= N(\mu_{f|y}, \Sigma_{f|y}) \\ \mu_{f|y} &= \mu_f + \Sigma_{fy} \Sigma_{yy}^{-1} (y - \mu_y) \\ \Sigma_{f|y} &= \Sigma_{ff} - \Sigma_{fy} \Sigma_{yy}^{-1} \Sigma_{fy} \end{aligned}$$

Two intuitive limits emerge from these formulas. If the data explains very little (Σ_{fy} small), the conditional variance approaches the prior variance Σ_{ff} — the data has not reduced our uncertainty. If the data explains a lot, the conditional variance shrinks to zero — we are confident, and the mean update is a linear function of the observations y .

GP regression: prediction at new points

For a test input y^* (with prior mean μ_{y^*} and covariance $\Sigma_{y^*y^*}$) the predictive distribution is again Gaussian:

$$\begin{aligned} p(y^* | y) &= N(\mu_{y^*|y}, \Sigma_{y^*|y}) \\ \mu_{y^*|y} &= \mu_{y^*} + \Sigma_{y^*y} \Sigma_{yy}^{-1} (y - \mu_y) \\ \Sigma_{y^*|y} &= \Sigma_{y^*y^*} - \Sigma_{y^*y} \Sigma_{yy}^{-1} \Sigma_{y^*y} \end{aligned}$$

Notice the symmetry with the conditioning formula — only the input set differs. This is what makes GPs so elegant: a single Gaussian conditioning rule serves both for fitting the latent function on the training data and for predicting at new points.

Concrete example

For $y(t) = t \cdot \sin(t)$ on $t \in [0, 10]$, we can sample functions from the GP prior, observe a couple of y values (say, at $t = 3$ and $t = 6$), and watch the posterior tighten dramatically around the observations while keeping wide error bars elsewhere. The shape of the kernel determines how quickly the uncertainty grows with distance from the data.

Take-home

A GP is just a multivariate Gaussian over function values at any finite set of inputs. Conditioning on observations gives closed-form posterior mean and variance — predictions are accompanied by principled uncertainty estimates without any backpropagation. The kernel choice is the modelling choice that matters most.

Lecture 5 — Gaussian Process Classification

Lecture 5 extends GPs from regression to classification. The challenge: outputs are now discrete labels $y \in \{-1, +1\}$, not continuous values, so the Gaussian conditioning that gave us closed-form regression no longer directly applies.

Regression vs classification

For regression we have $x \in \mathbb{R}^D$, $y_i \in \mathbb{R}$ and we model $y_i = f(x)$. For binary classification we have $x \in \mathbb{R}^D$, $y_i \in \{-1, +1\}$ and we want $p \in [0, 1]$ — the probability of class +1. We need a way to map a real-valued latent function to a probability.

Likelihood: Gaussian linear model + sigmoid

$$p(y = \pm 1 \mid x, w) = \sigma(x^T w), \text{ with } \pi(x) := \sigma(x^T w)$$

The sigmoid σ squashes $\mathbb{R}$ to $(0, 1)$. Two common choices are the logistic function $\lambda(z) = 1 / (1 + e^{-z})$ and the standard normal CDF $\Phi(z) = \int_{-\infty}^z N(x \mid 0, 1) dx$ (the probit). Because both are symmetric around 0, the likelihood for class $y_i \in \{-1, +1\}$ can be written compactly as:

$$p(y_i \mid x_i^T w) = \sigma(y_i \cdot f_i), \text{ with } f_i = x_i^T w \text{ (the latent GP)}$$

Posterior over weights

With a Gaussian prior $w \sim N(0, \sigma_0)$ (or $N(0, \Sigma_0)$), Bayes' rule gives the posterior

$$p(w \mid y, X) = p(y \mid X, w) \cdot p(w) / p(y \mid X), \text{ with } p(y \mid X) = \int p(y \mid X, w) p(w) dw.$$

Because the sigmoid breaks Gaussianity, this integral is no longer tractable in closed form. Approximations (Laplace approximation, expectation propagation, variational inference, MCMC) are the price of switching from regression to classification.

Two-step prediction

Practically, GP classification proceeds in two steps:

- **Step 1.** Place a Gaussian process prior on a latent function $f(x)$ — exactly as in GP regression.
- **Step 2.** Filter f through the sigmoid to obtain $\pi(x) = p(y = +1 \mid x) = \sigma(f(x))$.

For a test point x^* :

$$p(y^* = +1 \mid x^*, D) = \int p(y^* = +1 \mid w, x^*) p(w \mid D) dw$$

The posterior over weights is unimodal but not Gaussian; a Gaussian approximation is therefore the most common practical choice. The implementation is again two-step: first compute $p(f^* \mid X, y, x^*) = \int p(f^* \mid X, x^*, f) p(f \mid X, y) df$, then filter through the sigmoid to obtain the probabilistic prediction $\pi^* = \int \sigma(f^*) p(f^* \mid X, y, x^*) df^*$. The latent f acts as a nuisance variable — we never observe it, only the resulting class probabilities.

Take-home

GP classification = GP regression on a latent function, squashed by a sigmoid. The clean closed-form math of GP regression breaks because of the non-Gaussian likelihood; in practice one uses the Laplace, EP, or variational approximation. The intuition (smooth latent function, kernel-based similarity, principled uncertainty) carries over.

Lecture 6 — State Space Models & Kalman Filtering

State space models (SSMs) describe systems where a hidden state evolves in time and is observed only indirectly through noisy measurements. The Kalman filter is the closed-form Bayesian filter for the special case of *linear-Gaussian* SSMs; the Extended and Unscented Kalman filters extend it to non-linear systems.

State Space Models (SSM)

An SSM provides a general framework to describe deterministic and stochastic dynamical systems observed indirectly through a stochastic process. The term originated in control engineering (Kalman, 1960). Applications include object tracking (pedestrians, vehicles in self-driving cars), GPS navigation, aerospace engineering, surveillance and finance.

Notation: $z_n \in \mathbb{R}^D$ is the latent state at time n , $y_n \in \mathbb{R}^L$ the corresponding observation. The model has two pieces:

$$\begin{aligned} z_n &= f(z_{n-1}, u_{n-1}, r_n) \leftarrow \textbf{transition model} \\ y_n &= h(z_n, u_n, q_n) \leftarrow \textbf{measurement model} \end{aligned}$$

u_n is an optional deterministic input (e.g. a control signal), r_n is the system (process) noise and q_n is the observation noise. The belief state is estimated recursively starting from a specified initial state z_0 .

Linear-Gaussian SSM (LG-SSM)

An LG-SSM (also called a linear dynamical system) is the special case where both f and h are linear and the noises are Gaussian:

$$\begin{aligned} z_n &= F z_{n-1} + r_n, r_n \sim N(0, R) \\ y_n &= H z_n + q_n, q_n \sim N(0, Q) \end{aligned}$$

Equivalently: transition density $p(z_n | z_{n-1}) = N(F z_{n-1}, R)$ and observation density $p(y_n | z_n) = N(H z_n, Q)$. We assume F , H and the noise covariances are known.

Three related inference problems exist depending on the index k of the state we want to estimate given observations $y_{1:n}$:

- **Prediction** if $k > n$ (estimate future states)
- **Filtering** if $k = n$ (estimate the current state — the focus of KF)
- **Smoothing** if $k < n$ (estimate past states with all available data)

Kalman Filter (KF)

The Kalman filter is the algorithm that performs **exact Bayesian filtering** for an LG-SSM. It exploits the Markov property $p(z_n | z_{1:n-1}, y_{1:n-1}) = p(z_n | z_{n-1})$ and the conjugacy of Gaussians to keep the filtering density Gaussian at every time step. It alternates two steps:

- **1. Prediction (time update).** Use the transition model to propagate the previous filtering density forward in time. The predictive density is Gaussian with mean $F \mu_{n-1}$ and covariance $R + F \Sigma_{n-1} F^T$.
- **2. Correction (measurement update).** Combine the predictive density with the new observation y_n using Bayes' rule. The result is a Gaussian filtering density.

$$\begin{aligned}\mu_n &= \hat{\mu}_n + K_n (y_n - H \hat{\mu}_n) \\ \Sigma_n &= (I - K_n H) \Sigma_n \\ K_n &= \Sigma_n H^T (H \Sigma_n H^T + Q)^{-1} \text{ (Kalman gain)}\end{aligned}$$

The Kalman gain K_n balances trust in the prediction against trust in the measurement: larger when measurement noise Q is small and smaller when prediction uncertainty Σ_n is small.

Concrete example: tracking a biker

Estimate the position of a cyclist on a straight 1D road from noisy antenna measurements of distance and velocity. The state is $z = [\text{position}; \text{velocity}]$; the transition model is a constant-velocity model $F = \begin{bmatrix} 1, \Delta t \\ 0, 1 \end{bmatrix}$ with Gaussian process noise; the measurement model H selects what we observe. R and Q must be tuned — in practice a search over these parameters is performed.

Extended Kalman Filter (EKF)

When the transition or measurement models are non-linear, the transition and predictive distributions are no longer Gaussian and the integrals no longer tractable. The EKF approach is to **linearise around the current state estimate** using a first-order Taylor expansion:

$$\begin{aligned}z_n &\approx f(\mu_{n-1}) + F (z_{n-1} - \mu_{n-1}) + r_n \\ y_n &\approx h(\hat{\mu}_n) + \hat{H} (z_n - \hat{\mu}_n) + q_n\end{aligned}$$

F and $\hat{H}$ are the Jacobians of f and h with respect to z . With the linearised model the standard KF equations apply. The cost is approximation error which grows when the functions are highly non-linear or when prior uncertainty is large.

Unscented Kalman Filter (UKF)

The EKF performs poorly when (a) the prior covariance is large or (b) the system is highly non-linear near the current mean. The UKF uses a deterministic **unscented transformation** to address both:

- Pick a minimal set of weighted **sigma points** $z^{(l)}$ that capture the mean and covariance of the prior.
- Propagate each sigma point through the non-linear function f .
- Recompute the new mean and covariance from the propagated points: $\mu \approx \Sigma w_l z^{(l)}$ and $\Sigma \approx \Sigma w_l (z^{(l)} - \mu)(z^{(l)} - \mu)^T$.

Compared to the EKF, the UKF does not approximate the function (no Jacobian needed) but estimates the resulting Gaussian distribution directly from samples — typically more accurate.

Critical comparison

Estimator	Model assumptions	Noise	Cost
Kalman Filter (KF)	Linear	Gaussian	Low
Extended KF (EKF)	Non-linear (locally linearised)	Gaussian	Low / Medium (Jacobians)
Unscented KF (UKF)	Non-linear	Gaussian	Medium

Take-home

If your system is linear-Gaussian, the Kalman filter is exact and cheap — use it. If the dynamics are non-linear but smooth and the prior is tight, EKF (linearisation by Jacobian) is fine. If the system is strongly non-linear or your uncertainty is wide, the UKF (sigma-point propagation) usually wins. All three still assume Gaussian noise — for non-Gaussian, see particle filtering (Lecture 7).

Lecture 7 — Sequential Monte Carlo & Particle Filtering

The Kalman family assumes Gaussian noise. When the noise is non-Gaussian or the dynamics are strongly non-linear, we need a more flexible approach. **Particle filtering** is a Monte Carlo method that represents the filtering density by a weighted cloud of samples (particles) and propagates it through time.

Recap: Bayesian filtering

Given the filtering density at step $n - 1$, the update has the same two steps as before:

$$\begin{aligned} \text{Prediction: } p(z_n | y_{1:n-1}) &= \int p(z_n | z_{n-1}) p(z_{n-1} | y_{1:n-1}) dz_{n-1} \\ \text{Correction: } p(z_n | y_{1:n}) &= p(y_n | z_n) p(z_n | y_{1:n-1}) / p(y_n | y_{1:n-1}) \end{aligned}$$

When variables are Gaussian and transitions linear, this collapses to the Kalman filter. In general the integrals are intractable.

Monte Carlo essentials

We want to estimate $\mathbb{E}[f] = \int f(z) p(z) dz$. If we can sample $z^{(s)} \sim p(z)$, the law of large numbers gives $\mathbb{E}[f] \approx (1/S) \sum f(z^{(s)})$. Three core techniques solve the common problems:

- **Rejection sampling.** Draw from a proposal $q(z)$, accept with probability proportional to $\tilde{p}(z) / (k q(z))$. Wasteful when k is large.
- **Importance sampling.** Draw from $q(z)$ and reweight: $\mathbb{E}[f] \approx (1/S) \sum f(z^{(s)}) \cdot p(z^{(s)}) / q(z^{(s)})$. No samples are rejected; weights p/q encode importance.
- **Sampling-importance-resampling (SIR).** Sample from q , compute weights, then resample with replacement from the discrete distribution of weights. The resulting samples are approximately distributed as p ; the approximation becomes exact as $S \rightarrow \infty$.

Particle filtering (PF)

Particle filtering applies sequential importance sampling to the filtering problem. Each particle is a state hypothesis with an associated weight. The filtering density at time n is approximated by:

$$p(z_n | y_{1:n}) \approx \sum_{s=1..S} \tilde{w}_n^{(s)} \cdot \delta(z_n - z_n^{(s)})$$

Each step, the weights are updated recursively from a Markov property argument:

$$w_n^{(s)} = w_{n-1}^{(s)} \cdot p(y_n | z_n^{(s)}) p(z_n^{(s)} | z_{n-1}^{(s)}) / q(z_n^{(s)} | z_{1:n-1}^{(s)}, y_{1:n})$$

The degeneracy problem and resampling

A naive sequential importance sampling fails after a few steps because most particles end up with negligible weights. The **effective sample size** $S_{\text{eff}} = 1 / \sum_s (w_n^{(s)})^2$ quantifies the problem. When S_{eff} drops below a threshold, we resample: discard particles with low weights, replicate those with high weights, reset all weights to $1/S$. The resulting set is i.i.d. from the discrete approximation.

Concrete example: cyclist on a hill

Estimate the horizontal position of a cyclist on a known terrain profile from noisy altitude measurements. Each particle is a position hypothesis. At each step we (1) propagate particles through the transition model $z_{n+1}^{(s)} = f(z_n^{(s)}) + \varepsilon$, (2) update weights using the new measurement: $w_{n+1}^{(s)} = w_n^{(s)} \cdot p(y_{n+1} | z_{n+1}^{(s)})$, (3) resample if needed. After a few steps the cloud concentrates around the most plausible hypothesis.

Pros and cons

Pros	Cons
Represent arbitrary densities (multi-modal, heavy-tailed)	High computational complexity
Adaptively focuses on probable state-space regions	Choosing the number of particles is hard
Handles non-Gaussian noise	Particle count grows quickly with state dimension
Recursive Bayesian inference for non-linear systems	Risk of degeneracy without resampling

Take-home

Particle filtering trades closed-form math for sample-based generality. It can represent any distribution but pays the price in computation, and you must monitor the effective sample size and resample to avoid degeneracy. It is the default tool when the Kalman family's Gaussian assumptions break.

Lecture 8 — Autoregressive Models (AR / MA / ARMA / ARIMA)

Lecture 8 introduces the workhorse linear models of classical time-series analysis: autoregressive (AR), moving average (MA), and their combinations ARMA and ARIMA. These are simple, interpretable, parametric models that often work surprisingly well on short-horizon forecasts.

Building blocks

- **White noise.** A sequence $\{e_t\}$ of zero-mean, constant-variance σ^2 , uncorrelated random variables. If $e_t \sim N(0, \sigma^2)$ we call it Gaussian white noise.
- **Random walk.** $y_t = y_{t-1} + e_t$; the path is the cumulative sum of white noise increments.

Linear process representation

A linear process can be written as an infinite moving average of past white noise:

$$y_t = \mu + e_t + \psi_1 e_{t-1} + \psi_2 e_{t-2} + \dots = \mu + \Psi(q^{-1}) e_t$$

where q^{-1} is the backshift operator ($q^{-k} e_t = e_{t-k}$) and $\Psi(q^{-1}) = 1 + \psi_1 q^{-1} + \psi_2 q^{-2} + \dots$ is a linear filter. If $\sum |\psi_i| < \infty$, the filter is stable and the process is stationary. Equivalently, the same process can be written as an infinite autoregression: $\Pi(q^{-1}) y_t = \mu + e_t$. These two formulations are algebraically equivalent.

Autoregressive models AR(n)

$$AR(n): y_t = a_1 y_{t-1} + a_2 y_{t-2} + \dots + a_n y_{t-n} + e_t$$

n is the order — how far back to look. Compactly: $A(q^{-1}) y_t = e_t$, where $A(q^{-1}) = 1 - a_1 q^{-1} - \dots - a_n q^{-n}$ is the autoregressive operator. Special cases:

- **AR(0)** = white noise.
- **AR(1):** $y_t = a_1 y_{t-1} + e_t$. If $|a_1| \rightarrow 0$ the process looks like white noise; if $a_1 < 0$ it oscillates around zero; if $a_1 = 1$ it becomes a random walk.

Choosing n : look at the partial autocorrelation function (PACF). For an AR(n) process the theoretical PACF cuts off at lag n .

Moving average models MA(m)

$$MA(m): y_t = e_t + b_1 e_{t-1} + \dots + b_m e_{t-m}$$

Here the value of the series is a linear combination of m past random shocks. Compactly: $y_t = B(q^{-1}) e_t$. **Choosing m :** look at the autocorrelation function (ACF) — for an MA(m) process the theoretical ACF cuts off at lag m .

AR vs MA — critical comparison

- AR models ignore correlated noise structures; MA models do not.
- Finite MA models are **always** stationary; finite AR models may not be (e.g. AR(1) with $a_1 = 1$ is a random walk).
- All finite AR(n) processes can be expressed as infinite MA processes, and all finite invertible MA(m) processes as infinite AR processes.
- Parameter estimation for MA is generally harder than for AR (the likelihood is non-linear in the parameters).

ARMA(n, m): combining AR and MA

$$\text{ARMA}(n, m): y_t = \sum a_i y_{t-i} + \sum b_j e_{t-j} + e_t$$

$$\text{or } A(q^{-1}) y_t = B(q^{-1}) e_t$$

Hyperparameters n and m are inspected via the ACF / PACF jointly: AR(n) → PACF cuts off at n, ACF tails off; MA(m) → ACF cuts off at m, PACF tails off; ARMA(n, m) → both tail off. The choice is not unique.

Non-stationary series → ARIMA

ARMA assumes stationarity. Real series often have a non-stationary trend μ_t plus a zero-mean stationary part ϕ_t : $y_t = \mu_t + \phi_t$. We *stationarise* the series two ways:

- **Detrending.** $\hat{y}_t = y_t - \hat{\mu}_t$. Requires estimating the trend.
- **Differencing.** $\nabla y_t = y_t - y_{t-1}$, or in operator form $\nabla = 1 - q^{-1}$. Higher-order differencing $\nabla^d = (1 - q^{-1})^d$. No parameters to estimate — a clean way to remove trends and seasonality.

$$\text{ARIMA}(n, d, m): A(q^{-1}) \nabla^d y_t = B(q^{-1}) e_t$$

An ARIMA(n, 0, m) is the same as ARMA(n, m). The general ARIMA workflow: (1) check stationarity / seasonality, difference if needed; (2) identify (n, d, m); (3) estimate parameters by least squares or MLE; (4) validate by residual analysis; (5) forecast.

ARIMA pros and cons

Pros	Cons
Effective for short-term forecasting (e.g. inflation forecasts)	Identifying the correct model is hard and computationally expensive
Parametric — works with relatively small datasets	Poor at predicting series with turning points

Take-home

Reach for ARIMA when your data is roughly stationary (after a few differences), the dataset is small, and the forecast horizon is short. Use ACF / PACF as your diagnostic compass for choosing (n, d, m). For long-horizon, non-linear or large-data problems, deep learning models (L10–13) usually do better.

Lecture 9 — Data Mining with Time Series

Data mining is the art of finding non-obvious patterns in large, complex, possibly heterogeneous datasets. Time-series data mining borrows tools from statistics, information retrieval, machine learning and visualisation. This lecture covers the four most useful workhorses: data preprocessing, frequency analysis, dynamic time warping and advanced feature extraction.

Preprocessing: reduction and transformation

- **Sampling** — reduce the number of observations.
- **Selection / projection** — reduce the number of features.
- **Discretisation / aggregation** — reduce the number of distinct values.
- **Normalisation.** Mean normalisation: $s' = (s - \mu) / (s_{\max} - s_{\min})$. Min-max normalisation: $s' = (s - s_{\min}) / (s_{\max} - s_{\min}) \rightarrow [0, 1]$.
- **Z-score standardisation.** $s' = (s - \mu) / \sigma \rightarrow$ zero mean, unit variance. For Gaussian data the values mostly land in $(-3, 3)$.

Frequency analysis

Many time series exhibit periodic behaviour. Spectral analysis transforms data from time to frequency domain via the **Fourier transform**:

$$S_t = \sum_k [a_k \sin(2\pi k t / N) + b_k \cos(2\pi k t / N)]$$

Different signals have different Fourier coefficients, allowing comparison in frequency space. The **spectral density** represents the covariance of the series and is estimated by the periodogram — the squared correlation between the series and sine/cosine waves.

Continuous Wavelet Transform (CWT) extends the Fourier approach to non-stationary signals. It transforms a 1D series into a 2D scalogram (time $\times$ scale) and is robust to noise. Wavelets are particularly effective for damping identification in oscillating signals and for distinguishing between different production regimes or fault types in industrial machinery.

Dynamic Time Warping (DTW)

Euclidean distance fails for series that are similar in shape but shifted or stretched in time. DTW finds the optimal global alignment between two series. The algorithm:

- Initialise a distance matrix M .
- Fill from bottom-left using the recurrence $M[i, j] = \text{dist}(x_i, y_j) + \min(M[i-1, j-1], M[i, j-1], M[i-1, j])$.
- Trace the warping path d from the top-right corner.
- The total distance is $D = \sum \text{dist}(i_k, j_k)$ along the path.

Pros: non-linear time distortion finds non-trivial similarities (speech recognition, signature recognition, gait analysis). **Cons:** $O(N \cdot M)$ computational cost; needs a reliable reference template per class.

Matrix profile

The matrix profile is a powerful, domain-agnostic data structure. For a sliding window of size m on a series of length n , it stores two vectors: a **distance profile** (minimum z -normalised Euclidean distances) and a **profile index** (location of the first nearest neighbour). The algorithm uses an exclusion zone to ignore trivial matches. Once computed, the matrix profile reveals:

- **Motifs** — repeated patterns (low values in the profile).
- **Discords** — anomalies (high values in the profile).

Advantages: efficient, exact, only one parameter (the window size m).

Signature method

From rough path theory: given a multivariate time series $X: [a, b] \rightarrow \mathbb{R}^D$, the **signature** is the infinite series of all iterated integrals of the path. The single-iterated integral is $S_{a,b}^i(X) = \int_{a \leq u \leq b} dX_u^i = X_b^i - X_a^i$; double-iterated integrals capture pairwise interactions, and so on. The signature is a powerful feature representation used in Chinese character recognition and in financial data streams.

Python tooling

- **tsfresh** — automatic feature extraction from time series (≈ 800 features).
- **TSFEL** — feature extraction in time, statistical and spectral domains.
- **matrixprofile**, **stumpy** — Python implementations of the matrix profile.
- **pyts** — classical time-series classification methods.
- See the systematic review by Siebert et al. (2021) for an overview.

Take-home

Before reaching for a deep model, run a frequency analysis and a DTW / matrix-profile pass. They are cheap, interpretable, and frequently uncover the structure that turns a hard ML problem into a trivial one. tsfresh + a tree-based classifier is a remarkably strong baseline for time-series classification.

Lecture 10 — Deep Learning: Recurrent Models (RNNs)

Lecture 10 begins the deep-learning block with a quick refresher on neural networks (perceptron → MLP → backprop) before introducing recurrent neural networks, the natural architecture for sequence modelling, and their training algorithm Backpropagation Through Time (BPTT).

Why deep learning?

Classical methods relied on hand-crafted features (MFCCs for speech, i-Vectors for speaker recognition, SIFT for vision). These require expert domain knowledge and break down when no general feature exists or the input is very high-dimensional. Deep learning lets the model discover features in a data-driven way and frequently beats prior approaches (e.g. YOLO for object detection).

From the perceptron to the MLP

The perceptron is a computational model of a neuron: weighted inputs are summed, a bias is added, and the result is squashed by an activation function:

$$y = \sigma(\sum_i w_i x_i + b)$$

Common activations: **sigmoid** $1 / (1 + e^{-x})$, **tanh(x)**, and **ReLU(x) = max(x, 0)**. Stacking many perceptrons in parallel forms a layer $y = \sigma(Wx + b)$; chaining many layers gives a multilayer perceptron (MLP). Layers between input and output are called **hidden** layers; an MLP with one hidden layer is already a universal function approximator.

Loss, gradient descent, backpropagation

The loss function compares predicted $\hat{y}$ with target y . Two common choices: $MSE = (1/n) \sum (\hat{y}_i - y_i)^2$ for regression, and $CE = - \sum_i y_i \log \hat{y}_i$ for classification.

Gradient descent updates parameters by stepping in the direction that reduces the loss: $\theta^{(i+1)} = \theta^{(i)} - \lambda \nabla \theta$. **Backpropagation** efficiently computes $\nabla \theta$ for all weights at once by applying the chain rule layer by layer, from output to input — computing each gradient only once and reusing intermediate results.

Why feed-forward fails for sequences

- Cannot work online — the whole sequence has to be fed at once.
- Considers only the current input — no memory of previous time steps.
- Inefficient and unable to handle variable-length sequences.

The recurrent neural network (RNN)

An RNN is a neural network with feedback connections: activations flow in a loop, allowing temporal processing. Components: input x_t , hidden state h_t , neural-network block A , output y_t . Mathematically:

$$h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t)$$

$$y_t = \sigma(W_{hy} h_t)$$

More generally $h_t = f_{\theta}(h_{t-1}, x_t)$ and $y_t = f_{\phi}(h_t)$. The order of the dynamical system equals the dimension of the hidden state h_t . A universal-approximation theorem holds: any non-linear dynamical system can be approximated by a sufficiently large RNN with sigmoidal hidden units.

RNN architectures

Pattern	Use case
one-to-one	Classical feedforward, no time
one-to-many	Image captioning (one image → caption sequence)
many-to-one	Sentiment analysis (review → label)
many-to-many (synced)	Named-entity recognition, frame-level tagging
many-to-many (offset)	Machine translation (encoder-decoder)

Backpropagation Through Time (BPTT)

To train an RNN, unfold it through time into a deep feed-forward computation graph and apply standard backpropagation. The procedure:

- **1.** Forward propagate the entire sequence and store all activations.
- **2.** Compute the per-time-step loss $\mathcal{L}^{(t)}$ and the total loss $\mathcal{L} = \sum \mathcal{L}^{(t)}$.
- **3.** Backpropagate gradients through the unfolded graph and update weights using the learning rate.

Cost. BPTT is computationally expensive because the unfolded graph is as deep as the sequence is long. **Truncated BPTT** trims this by performing the forward / backward pass on a fixed-length sub-sequence, accepting an approximation in exchange for tractable training.

Pros and cons

Pros	Cons
Same input size regardless of sequence length	Vanishing / exploding gradients on long sequences
Variable-length sequences are natural	Sequential computation → slow training
Same transition reused at every time step	Hard to learn very long-range dependencies

Take-home

RNNs add memory to the perceptron family by letting activations loop through time. Training uses backpropagation on the unfolded graph (BPTT). The fundamental weakness — vanishing / exploding gradients — is what motivates the gated architectures of the next lecture.

Lecture 11 — Gated Models (LSTM, GRU)

Vanilla RNNs struggle to learn dependencies over long time horizons. Gated architectures — most famously the LSTM and the GRU — solve this by adding internal switches that control what is remembered, forgotten, and exposed to the next step.

Long-term dependencies

A task displays **long-term dependencies** when the prediction at time t depends on input presented at a much earlier time $\tau \ll t$. RNNs are theoretically capable of learning such dependencies, but their parameters tend to settle in sub-optimal solutions that focus on short-term structure.

A parametric dynamical system that can store relevant state information must satisfy three properties:

- **Latching:** store information for an arbitrary duration.
- **Robustness:** resist noise that is irrelevant to the prediction.
- **Learnability:** all parameters must be trainable.

The vanishing / exploding gradient problem

When a single RNN cell tries to latch one bit of information using a hyperbolic attractor, two failure modes arise: either the system is very sensitive to noise, or the gradients of the cost with respect to the early activations *vanish exponentially*.

- **Exploding gradient.** Gradients become very large → unstable learning. Mitigated by **gradient clipping**: cap the gradient norm at a threshold during backpropagation.
- **Vanishing gradient.** Gradients become very small → learning stalls on long-range dependencies. Mitigated by introducing **shortcuts** in the architecture, i.e. **gates**.

Long Short-Term Memory (LSTM)

Hochreiter and Schmidhuber's LSTM (1997) extends the RNN cell with a second state — the **cell state** c_t — that flows through the network with only minor interactions, providing a near-uninterrupted channel for information. Three gates regulate this flow:

$$\begin{aligned}
 f_t &= \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \leftarrow \textbf{forget: what to drop from } c_{t-1} \\
 i_t &= \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \leftarrow \textbf{input: how much new info to admit} \\
 g_t &= \tanh(W_g \cdot [h_{t-1}, x_t] + b_g) \leftarrow \textbf{candidate values} \\
 c_t &= f_t \odot c_{t-1} + i_t \odot g_t \leftarrow \textbf{cell update} \\
 o_t &= \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \leftarrow \textbf{output gate} \\
 h_t &= o_t \odot \tanh(c_t)
 \end{aligned}$$

From the original 1997 paper: each cell's internal architecture guarantees a *constant error carousel* within its cell state — the basis for bridging very long time lags. The forget gate decides how much of the previous cell state to keep; the input gate (combined with the candidate values) decides what new information to add; the output gate decides what part of the cell state to expose as the new hidden state.

Gated Recurrent Unit (GRU)

Introduced in 2014, the GRU is a streamlined LSTM with only two gates (**reset** and **update**) and a single state — no separate cell state. Fewer parameters and faster training. An empirical study by Jozefowicz et al. (2015) found GRU outperformed LSTM on most tasks except language modelling. The choice between them depends on the data; empirical comparison is the only reliable guide.

Phased-LSTM

Phased-LSTM (Neil et al., 2016) extends the LSTM with an additional **time gate** controlled by a parametrised oscillation. The gate opens only during a small fraction of each cycle, naturally handling inputs that arrive at asynchronous times — particularly useful for event-based sensors.

Take-home

If your sequence has long-range dependencies, a vanilla RNN will likely fail. The LSTM solves the vanishing-gradient problem with three gates and a protected cell state. GRUs trade a slight loss in expressive power for fewer parameters and faster training. Phased-LSTM is the go-to choice for irregularly sampled or event-driven data.

Lecture 12 — Convolutional Models for Time Series

Convolutional Neural Networks (CNNs), most famous in computer vision, are increasingly applied to sequential data. They sidestep two pain points of RNN/LSTM: poor parallelism and difficult long-sequence training.

Inspiration: the visual cortex

Cortical neurons in the visual cortex respond only to small portions of the visual field (small receptive fields). Small fields are stimulated by high spatial frequencies (fine details), large fields by low spatial frequencies (coarse details). CNNs mirror this with sparse, localised connectivity implemented through convolutions.

The convolution operation

$$(input * filter)[n] = \sum_{m=-\infty}^{+\infty} input[m] \cdot filter[n - m]$$

In 2D, a small filter (or kernel) slides across the input. Each output value is the inner product between the kernel and the corresponding input patch — only a small subset of inputs influences each output. Multiple filters can run in parallel in a single convolutional layer; their outputs concatenate into a multi-channel feature map.

Building a CNN

- **Convolutional layers** stack to learn increasingly abstract features. Many filters per layer; multi-channel feature maps.
- **Pooling.** Reduces spatial dimensions to keep computation manageable. Max-pooling: $y = \max$ of a 2×2 patch. Other variants: mean, L2-norm. No general consensus on which is best — validate empirically.
- **Padding.** Convolution shrinks the output: an input of size n with a filter of size m produces a feature map of size $n - m + 1$. Adding zeros around the input (zero-padding) preserves spatial dimensions.

Translation equivariance. CNNs are translation equivariant: if the input is shifted by k pixels, the feature map is shifted by k as well. The same kernel scans the whole image, so a pattern is recognised regardless of its location — central to why CNNs work for object recognition.

1D CNNs for time series

$$y_t = w_1 x_t + w_2 x_{t+1} + w_3 x_{t+2}$$

A 1D convolution slides a small kernel along the temporal axis. Compared to RNNs, 1D CNNs are easy to parallelise, scale to large inputs and have shorter effective gradient paths. Caveat: a standard convolution can look at **future** values relative to the current one, which violates causality. RNNs respect causality by construction; CNNs need extra care (see TCN below).

ConvLSTM

A recurrent layer like the LSTM, but the matrix multiplications inside the LSTM cell are replaced by convolutional operations. Suited for video processing — the network preserves the spatial layout of its input throughout time. Note: not the same as a CNN+LSTM stack, where a CNN first extracts a flat embedding from each frame and the LSTM consumes the embeddings.

ConvLSTM has been applied to gesture recognition, precipitation nowcasting, medical image segmentation, and salient-object detection in video.

Temporal Convolutional Networks (TCN)

Introduced in 2016 for video action segmentation, TCNs use an encoder-decoder framework and accept a sequence of any length, outputting a sequence of the same length. The defining feature is the **causal convolution**: the output at time t is convolved only with observations up to time t — there is no information leakage from the future. For 2D data this is implemented as masked convolution; for 1D audio data it is achieved by shifting the output of a normal convolution by a few timesteps.

TCNs combine the parallelism of CNNs with respect for causality, and have been shown to exhibit **longer effective memory than RNNs** given the same parameter budget. Applications include weather prediction, sound-event localisation and action segmentation.

FF-NN vs RNN vs CNN — critical comparison

Property	FF-NN	RNN	CNN
Data layout	Tabular	Sequential	Grids
Weight sharing	No	Yes	Yes
Recurrent connections	No	Yes	No
Translation equivariance	No	No	Yes
Parallel computation	Yes	No	Yes
Vanishing gradient	Yes	Yes	Yes
Spatial relationships	No	No	Yes
Causality respected	No	Yes	No (unless masked)

Take-home

1D CNNs and TCNs are competitive (often superior) alternatives to RNN/LSTMs for sequence modelling — faster to train, easier to parallelise, and capable of long memory. Use a TCN when causality matters; use ConvLSTM when both spatial and temporal structure are essential (video).

Lecture 13 — Attention-based Models & the Transformer

Attention mechanisms — and the Transformer architecture built on them — have replaced recurrent models in most NLP tasks and are increasingly applied to time-series data. The core insight: instead of compressing an entire sequence into a fixed-length vector, let the model attend directly to all relevant input positions when producing each output.

Sequence-to-sequence and the bottleneck

A seq2seq model maps an input sequence to an output sequence (e.g. for machine translation). Classical implementations use an encoder-decoder RNN: the encoder compresses the input into a single context vector (embedding), the decoder generates the output from it. Drawbacks:

- Fixed-length context cannot remember long sequences.
- Single hidden state cannot encode complex dependencies.

Attention mechanism

Attention (Bahdanau et al., 2014) replaces the single context vector with a weighted sum of all encoder hidden states. For decoder position t , the context is:

$$c_t = \sum_{i=1..n} a_{t,i} \cdot h_i, \text{ with } a_{t,i} = \frac{\exp(\text{score}(s_{t-1}, h_i))}{\sum_j \exp(\text{score}(s_{t-1}, h_j))}$$

$h_i = [h_i^{\rightarrow}, h_i^{\leftarrow}]$ are the bidirectional encoder states; s_t is the decoder state. Attention can be characterised by:

- **Self-attention** (intra-attention): relates positions within the same sequence to build a representation of itself.
- **Soft vs hard**. Soft attention learns weights over all parts; hard attention picks one part at a time.
- **Global vs local**. Global attention attends over the whole sequence; local attention restricts to a window centred on the current position.

The Transformer architecture

Attention is all you need (Vaswani et al., 2017) introduced the Transformer, an architecture built entirely on self-attention with no recurrence or convolution. Components:

- **Positional encoding** — without recurrence, the model would be order-blind. Sinusoidal encodings inject position information by adding to the input embeddings.
- **Multi-head self-attention** based on Query (Q), Key (K), Value (V) matrices.
- An **encoder-decoder** architecture, both made of N stacked identical layers.

Multi-head self-attention

From each input vector, three vectors are derived by multiplying with learned matrices W^Q, W^K, W^V :

$$Q = X W^Q, K = X W^K, V = X W^V$$

$$\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_k}) \cdot V$$

The $\sqrt{d_k}$ scaling prevents softmax from saturating for large d_k . **Multi-head** attention runs h such operations in parallel with different learned projections, then concatenates the results — a single head finds one alignment, multiple heads find complementary ones:

$$\text{MultiHead}(Q, K, V) = \text{concat}(\text{head}_1, \dots, \text{head}_h)$$

$$\text{head}_i = \text{Attention}(Q W_i^Q, K W_i^K, V W_i^V)$$

Positional encoding

$$PE_{(pos, 2i)} = \sin(pos / 10000^{2i/d})$$

$$PE_{(pos, 2i+1)} = \cos(pos / 10000^{2i/d})$$

Each dimension uses a sinusoid of a different frequency. Positional encoding is added to the input embeddings of both encoder and decoder.

Encoder, decoder and decoding

Each encoder layer = (multi-head self-attention) + (positional FF-NN) + (residual connections + layer normalisation). Each decoder layer adds a **masked** first attention (preventing information leakage from future positions). Decoding works as follows: (1) the top encoder output is transformed into K and V matrices, (2) used by every decoder layer to attend over the encoded representation, (3) the process repeats until a special <end> symbol is generated.

Pros and cons

Pros	Cons
Direct access to far-away input parts	Quadratic time and memory complexity
Many degrees of freedom → captures complex dependencies	Data-hungry — needs large datasets
Highly parallel — strong on modern hardware	Tricky to train (learning-rate schedules, custom data loaders)
State of the art in many NLP tasks	Time-series research with Transformers is still maturing

Other attention-based approaches

- **Neural Turing Machines** — couple a neural network with an external memory and use content-based attention to mimic Turing machine read/write operations.
- **Pointer networks** — solve problems where outputs are positions in the input sequence; attention picks one input element as the output at each decoder step.

Take-home

Self-attention lets the model directly access any position of the sequence and is fully parallelisable. The Transformer is now the default architecture for sequence learning when data is plentiful and compute is available. For small / mid-size time-series problems, TCN/LSTM still often win because Transformers are data-hungry and training is finicky.

Lecture 14 — Domain Adaptation

All previous lectures assumed training and test data are drawn from the same distribution. In practice this is rarely true. **Domain adaptation** studies what to do when training distribution $Q \neq$ test distribution P , and how to design models that generalise to unseen domains.

Why the i.i.d. assumption breaks

A classic example: train an image classifier on photos taken with a professional camera and deploy it on smartphone images. The training and test marginal distributions over images differ — $Q_{X,Y} \neq P_{X,Y}$ — and the classifier degrades. Three terms of art:

- **Source domain** $Q_{X,Y}$ — labelled distribution used for training.
- **Target domain** $P_{X,Y}$ — different but related distribution where the model must perform well.
- **Domain shift** — the statistical difference between source and target.

Concrete examples: training on synthetic images and deploying on real ones; same scene captured in different seasons.

Three flavours of domain adaptation

Setup	Source samples	Target samples
Unsupervised DA	Labelled	Only unlabelled samples
Semi-supervised DA	Labelled	Unlabelled + a few labelled samples
Domain generalisation	Multiple labelled source domains	No samples available during training (a.k.a. OOD generalisation)

Note: in all three setups, evaluation requires labelled samples from the target domain (the target is *not* the same as the test set from previous lectures).

Unsupervised domain adaptation (UDA)

Given source samples $(X^S, Y^S) \sim Q_{X,Y}$ and target samples $X^T \sim P_X$, find $h \in H$ that minimises the target risk $\epsilon_T(h) = \mathbb{E}_{(x,y) \sim P} [L(h(x), y)]$. Typical assumptions:

- **Covariate shift.** The conditional label distribution does not change between source and target: $P(y | x) = Q(y | x)$.
- **Similarity of distributions.** Source and target marginals should be similar — $Q_X \approx P_X$.
- **Small joint error.** If labelled samples were available, the joint error should be small.

H-divergence and the main result

The H-divergence measures how easily a hypothesis class can distinguish samples from two distributions:

$$d_H(Q_X, P_X) = 2 \cdot \sup_{h \in H} |p_Q(h(x) = 1) - p_P(h(x) = 1)|$$

It can be estimated by training a classifier to distinguish source from target samples — the lower the achievable classification error ϵ_{class} , the higher the divergence: $d_H = 1 - 2\epsilon_{\text{class}}$.

The **main result** bounds the target error in terms of source error and divergence (for $\text{VC}(H) = d$, with probability $1 - \delta$):

$$\epsilon_T(h) \leq \epsilon_S(h) + \frac{1}{2} d_{H\Delta H}(Q_X, P_X) + \epsilon_{\text{joint}}$$

(target error ≤ source error + divergence + joint error)

$H\Delta H$ is the symmetric difference hypothesis space — the set of disagreements between any two hypotheses in H . The bound has inspired many practical methods: classical (metric learning, sample reweighting, subspace alignment) and deep-learning-based.

Domain Adversarial Neural Networks (DANN)

Ganin & Lempitsky's idea: learn an embedding such that performance on the task is maximised *and* domain classification is **minimised**. The network is trained to fool a domain-classifier head — if the embedding is invariant to the domain, the divergence term in the bound shrinks.

Domain generalisation (OOD generalisation)

Here we have K labelled training domains $\{P^{(e)}\}_{e=1}^K$ and we want to find $h \in H$ that performs well on a brand-new, unseen domain $P^{(K+1)}$. Datasets: DomainNet (345 classes; clipart, real, sketch, infograph, painting, drawings) and PACS (7 categories; photo, painting, cartoon, sketch). Two methods:

- **Baseline (do nothing)** — minimise the average loss across the available training domains and hope for the best.
- **Invariant representations** — learn a representation ϕ that is invariant across domains. A DANN-style architecture combines a feature extractor ϕ , a label classifier $\omega \circ \phi$, and a domain classifier $c \circ \phi$; the loss is $L = \sum L(\omega \circ \phi(x), y) - \lambda \sum L(c \circ \phi(x), y)$; minimise with respect to (ω, ϕ) , maximise with respect to c .

Take-home

Whenever your model crosses a distribution boundary at deployment (synthetic→real, hospital A→hospital B, sensor v1→sensor v2), think domain adaptation. The main theoretical tool is the H-divergence; the main practical recipe is to learn domain-invariant features, often via an adversarial domain classifier (DANN). Domain generalisation pushes this idea further: prepare for an unseen target by learning across multiple source domains.

Course Synthesis & Cheat Sheet

When to reach for which model

Situation	Recommended approach
Stationary, short, parametric forecasting	AR / MA / ARMA / ARIMA (L8)
Need uncertainty estimates, small dataset	Bayesian linear regression (L3) or GP regression (L4)
Binary classification, principled uncertainty	GP classification (L5)
Linear-Gaussian dynamical system, online filtering	Kalman filter (L6)
Smooth non-linear dynamics, online filtering	EKF or UKF (L6)
Strongly non-linear / non-Gaussian filtering	Particle filter (L7)
Feature-rich classification with classical ML	tsfresh + tree model + DTW (L9)
Sequence learning, modest dataset	LSTM / GRU (L11) or 1D-CNN / TCN (L12)
Long-range dependencies, large dataset	Transformer (L13)
Distribution shift between training and deployment	Domain adaptation / DANN (L14)

Recurring themes across the course

- **Stationarity is the gateway.** Most classical results (CLT, ACF, ARMA) require it. When in doubt, difference or detrend.
- **Uncertainty is information.** Bayesian methods, GPs, Kalman/UKF and particle filters all give a posterior, not a point. Use it.
- **The bias-variance / capacity-data trade-off is everywhere.** ARIMA shines on small data, Transformers shine on large data, GPs sit in between with built-in regularisation.
- **Memory mechanisms are the hard part of sequence modelling.** RNN's vanishing gradient → LSTM/GRU's gates → TCN's dilated causal convolutions → Transformer's all-to-all self-attention.
- **Distribution shift is real.** Always ask whether your training data is representative of deployment. If not, plan for domain adaptation.

Suggested practice path

After reading this summary, the recommended hands-on practice for each lecture (in the companion code repository) is:

- **L1-L2:** simulate stationary and non-stationary processes; compute mean, variance, ACF; build a linear-regression forecaster.
- **L3:** implement Bayesian linear regression in NumPy; visualise posterior shrinkage as data accumulates.
- **L4-L5:** implement a GP regressor with RBF kernel; extend it to classification with a Laplace approximation.
- **L6:** code a 1D Kalman filter for tracking; an EKF for a non-linear pendulum; a UKF for the same.
- **L7:** implement a particle filter for the cyclist-on-a-hill example; experiment with degeneracy and resampling thresholds.
- **L8:** fit AR, MA, ARMA, ARIMA models with statsmodels on a real dataset (e.g. airline passengers).
- **L9:** compute spectrograms with FFT and CWT; implement DTW from scratch; explore matrix profiles with stumpy.
- **L10-L11:** implement an RNN, LSTM and GRU from scratch in PyTorch on a toy sequence task; reproduce vanishing-gradient phenomena.
- **L12:** build a 1D-CNN classifier on physiological signals (e.g. ECG); implement causal dilated convolutions for a TCN.
- **L13:** implement scaled dot-product and multi-head attention; build a small Transformer for a sequence prediction task.
- **L14:** implement a DANN with PyTorch's gradient-reversal layer for a synthetic-to-real domain adaptation experiment.

Final thought

Time-series ML is a layered subject: probability gives you the language, Bayesian inference gives you the principled framework, classical models give you interpretable baselines, and deep learning gives you the scale. The strongest practitioners move fluidly between layers — they use ARIMA when it works, GPs when uncertainty matters, and Transformers when data is plentiful. This summary has been written so you can do the same.